SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	N. Vinay Yadav	**Reg. No.:**	192472061
Section / Batch:	Slot B/2024	**Date:**	

Code of Conduct

I N.Vinay Yadav (192472061) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N.Vinay Yadav.

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Part of Speech Tagging	Morphological decomposition of analyzing, analysis, and analytical; identification of roots, affixes, inflectional and derivational forms, and normalization for indexing.	Q1
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q2
Counting Words, English Word Classes, Part of Speech Tagging	Morphology-based normalization of govern, government, and governance for topic modeling, clustering, and lexical standardization.	Q3
Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Morphological analysis of activate, activation, and reactivation; identification of derivational layers, word-class changes, and semantic shifts.	Q4
Rule-Based Part of Speech Tagging, English Word Classes, Part of Speech Tagging	Inflectional normalization of create, creates, and creating; extraction of root forms and classification of tense/aspect transformations.	Q5

Annexure A – SCENARIO BASED

1. A large-scale search engine indexing system processes user queries containing word variants such as “analyzing, “analysis”, and “analytical”. The system must improve retrieval accuracy by grouping semantically related terms under a unified representation. Design a morphological processing approach to decompose each input word into root and affixes, distinguish between inflectional and derivational forms, and normalize them for indexing. Apply the process to the given inputs and determine the root form along with the morphological structure of each word.

- Develop a Python program to implement a rule-based morphological processing system for the input words "analyzing", "analysis", and "analytical". The program should identify the root word and separate the corresponding prefixes or suffixes using suitable morphological rules, classify each word as an inflectional or derivational form, normalize all related variants to a common representation for indexing, and generate a structured report showing the original word, extracted root, identified affix(es), transformation type, and normalized output to support efficient search engine retrieval.

Question 1: Morphological Processing of “analyzing”, “analysis”, and “analytical”

Approach: Use rule-based suffix analysis to identify the likely lexical base, distinguish grammatical inflection from word-forming derivation, and map related forms to the indexing key “analyse”. The words are related but do not all share the same surface root: “analyzing” is an inflected form of “analyse”; “analysis” is a related noun with an irregular stem alternation; and “analytical” is an adjective derived from “analysis”.

Python Program:

```
words = ["analyzing", "analysis", "analytical"]
rules = {
   "analyzing": {
      "root": "analyze", "affix": "-ing",
      "type": "Inflectional (present participle/gerund)",
      "normalized": "analyz"
   },
   "analysis": {
      "root": "analysis", "affix": "related stem: analys-",
      "type": "Derivational/lexical variant (noun)",
      "normalized": "analyz"
   },
   "analytical": {
      "root": "analysis", "affix": "-ical",
      "type": "Derivational (noun to adjective)",
      "normalized": "analyz"
   }
}
```

```
    print(f'{"Word":<14} {"Root/Base":<12} {"Affix/Structure":<22} '
          f'{"Type":<43} Normalized')
    for word in words:
        item = rules[word]
        print(f'{word:<14} {item["root"]:<12} {item["affix"]:<22} '
              f'{item["type"]:<43} {item["normalized"]}')
```

Expected result:

Word: analyzing

Root/Base: analyze

Affi: -ing

Type: Inflectional

Normalized Output: analyz

Word: analysis

Root/Base: analysis

Affix: analys- (related stem)

Type: Derivational/Lexical Variant

Normalized Output: analyz

Word: analytical

Root/Base: analysis

Affix: -ical

Type: Derivational

Normalized Output: analyz

```
========== RESTART: C:/Users/Reddy/Documents/NLP ASSESSMENTS/Q1 co2.py =========
Word          Root/Base    Affix/Structure       Type                                    Normalized
analyzing     analyze      -ing                  Inflectional (present participle/gerund)    analyz
analysis      analysis     related stem: analys-  Derivational/lexical variant (noun)        analyz
analytical    analysis     -ical                 Derivational (noun to adjective)          analyz
```

Result: The words “analyzing”, “analysis”, and “analytical” are analyzed and normalized to a common representation using morphological rules.

2. An AI-based sentiment analysis platform receives product reviews containing words like "disagree", "agreement", and "agreeable". The system must interpret sentiment consistently despite variations in word formation. Develop a morphological parsing strategy to identify prefixes, suffixes, and base forms, while capturing how derivational changes influence meaning. Process the given inputs to extract roots and classify each transformation, ensuring accurate semantic interpretation across all word forms.

- Develop a Python program to create a morphological parser for the input words "disagree", "agreement", and "agreeable". The program should detect and separate prefixes, suffixes, and the base word using rule-based morphological analysis, examine how derivational modifications alter the meaning of the root word, classify each transformation according to its morphological type, and produce a comprehensive output containing the original word, identified prefix, root word, suffix, transformation category, semantic interpretation, and normalized base form for use in sentiment analysis applications.

Question 2: Morphological Parsing of "disagree", "agreement", and "agreeable"

Approach: Identify the base "agree", then detect the prefix "dis-" and the suffixes "-ment" and "-able". The prefix reverses or negates the meaning; "-ment" forms a noun describing an act or state; "-able" forms an adjective meaning capable of being agreed to or considered acceptable, depending on context.

Python Program:

```
words = ["disagree", "agreement", "agreeable"]
analysis = {
    "disagree": ("dis-", "agree", "", "Prefix derivation",
                 "not agree; express a different opinion"),
    "agreement": ("", "agree", "-ment", "Suffix derivation",
                  "the state or result of agreeing"),
    "agreeable": ("", "agree", "-able", "Suffix derivation",
                  "pleasant or acceptable; context dependent")
}
print(f'{"Word":<14} {"Prefix":<9} {"Root":<9} {"Suffix":<9} '
      f'{"Type":<20} Semantic interpretation')
for word in words:
    prefix, root, suffix, kind, meaning = analysis[word]
    print(f'{word:<14} {prefix:<9} {root:<9} {suffix:<9} '
          f'{kind:<20} {meaning}')
    print("  Normalized base form:", root)
```

Word: disagree

Prefix: dis-

Root: agree

Suffix: None

Transformation: Prefix derivation

Meaning: Not agree

Normalized Base: agree

Word: agreement

Prefix: None

Root: agree

Suffix: -ment

Transformation: Suffix derivation

Meaning: State or result of agreeing

Normalized Base: agree

Word: agreeable

Prefix: None

Root: agree

Suffix: -able

Transformation: Suffix derivation

Meaning: Pleasant or acceptable

Normalized Base: agree

```
========== RESTART: C:/Users/Reddy/Documents/NLP ASSESSMENTS/Q2 co2.py =========
Word        Prefix     Root       Suffix     Type               Semantic Interpretation
-------------------------------------------------------------------------------------------------------
disagree    dis-       agree      -          Prefix derivation  not agree; express a different opinion
  └ Normalized base form: agree

agreement   -          agree      -ment      Suffix derivation  the state or result of agreeing
  └ Normalized base form: agree

agreeable   -          agree      -able      Suffix derivation  pleasant or acceptable; context dependent
  └ Normalized base form: agree
```

Result: - The morphological parser successfully identifies the prefixes, roots, and suffixes of “disagree”, “agreement”, and “agreeable” and normalizes them to the base form “agree”.

3. A text analytics pipeline in a news aggregation system handles documents containing terms such as “govern”, “government”, and “governance”. The system must standardize these variations for topic modeling and clustering. Construct a morphology-based normalization mechanism that decomposes each word into its root and affixes, identifies derivational layers, and maps them to a common base form. Apply the approach to the input words and derive the normalized representation along with their morphological breakdown.

- Develop a Python program to implement a morphology-based normalization module for the input words "govern", "government", and "governance". The program should perform morphological decomposition to identify the root word and successive derivational suffixes, determine the derivational level associated with each word, normalize all variants to a common lexical representation, and generate a structured output displaying the original word, root form, detected affix(es), derivational hierarchy, normalized representation, and the final output used for topic modeling and document clustering.

Question 3: Morphology-Based Normalization of “govern”, “government”, and “governance”

Approach: Treat “govern” as the lexical base. “Government” is formed with the noun-forming suffix “-ment”, while “governance” is formed with “-ance”. Both are derivational forms and refer to related but not identical concepts. For topic modeling, map them to the common family key “govern” while retaining their original words and derivational levels.

Python Program:

```
words = ["govern", "government", "governance"]
analysis = {
    "govern": ("govern", "none", 0, "base verb"),
    "government": ("govern", "-ment", 1, "derived noun"),
    "governance": ("govern", "-ance", 1, "derived noun")
}
print(f'{"Word":<14} {"Root":<10} {"Affix":<10} '
      f'{"Derivation level":<18} {"Normalized"}')
for word in words:
    root, affix, level, description = analysis[word]
    print(f'{word:<14} {root:<10} {affix:<10} {level:<18} {root}')
    print("  Word class/structure:", description)
```

Expected Output

Word: govern

Root/Base: govern

Affix: None

Derivation Level: 0

Word Class: Base verb

Normalized Output: govern

Word: government

Root/Base: govern

Affix: -ment

Derivation Level: 1

Word Class: Derived noun

Normalized Output: govern

Word: governance

Root/Base: govern

Affix: -ance

Derivation Level: 1

Word Class: Derived noun

Normalized Output: govern

```
========== RESTART: C:/Users/Reddy/D
Word          Root       Affix      Derivation level   Normalized
------------------------------------------------------------------
govern        govern     none       0                  govern
  └─ Word class/structure: base verb

government    govern     -ment      1                  govern
  └─ Word class/structure: derived noun

governance    govern     -ance      1                  govern
  └─ Word class/structure: derived noun
```

Result: -

Thus, the program has successfully identified and isolated the linguistic components of each target word, mapping out their root structures, affixes, and morphological derivation levels.

4. A document classification system processes input words such as “activate”, “activation”, and “reactivation” to improve semantic grouping and indexing.
Construct a morphological parsing strategy to identify prefixes, suffixes, and root forms while distinguishing derivational layers. Analyze how prefix addition and suffix transformation affect meaning and word class. Apply the process to decompose each word and map them to a unified base representation. Derive the morphological structure and normalized root for each input word.

- Develop a Python program to implement a morphological parsing and normalization system for the input words "activate", "activation", and "reactivation". The program should detect and separate prefixes, root words, and suffixes using rule-based morphological parsing, identify the sequence of derivational transformations that produce each word, analyze the effect of each morphological modification on the word class and semantic meaning, and generate a structured report showing the original word, prefix (if present), root word, suffix, derivational sequence, normalized base form, and final parsed representation for use in document classification and semantic indexing.

Question 4: Morphological Parsing of “activate”, “activation”, and “reactivation”

Approach: Use “activate” as the base verb. “Activation” is formed by changing the verb into a noun with “-ion” (with the spelling/stem change activat- → activation). “Reactivation” adds the prefix “re-” to indicate doing the action again. Normalize all three to the base family “activate”, while preserving the derivational sequence and meaning.

Python Program:

```
words = ["activate", "activation", "reactivation"]
analysis = {
   "activate": ("", "activate", "", "activate",
                "verb; cause something to become active"),
   "activation": ("", "activat", "-ion", "activate",
                  "noun; the process or result of activating"),


 "reactivation": ("re-", "activat", "-ion", "activate",
"noun; the process of activating again")
}
print(f'{"Word":<16} {"Prefix":<8} {"Root/stem":<12} '
     f'{"Suffix":<8} {"Normalized":<12} Meaning')
for word in words:
   prefix, root, suffix, norm, meaning = analysis[word]
   print(f'{word:<16} {prefix:<8} {root:<12} {suffix:<8} '
         f'{norm:<12} {meaning}')
```

Expected Output: -

Word	Root/Base	Affix	Derivation Level	Word Class	Normalized Output
activate	active	-ate	1	Derived verb	active
activation	active	-ation	1	Derived noun	active
reactivation	active	re-ation	2	Derived noun	Active

```
========================================== RESTART: C:/Users/Reddy/Documents/NLP ASSESSMENTS/Q4 co2.py ==
Word           Prefix   Root/Stem    Suffix     Normalized   Meaning
-------------------------------------------------------------------------------------------
activate                active       -ate       active       verb; cause something to become active
activation              active       -ation     active       noun; the process or result of activating
reactivation   re-       active       -ation     active       noun; the process of activating again
```

Result: - The program successfully identified the morphological components of the target words.

It correctly broke down each term into its prefix, root/stem, and suffix variations.

5. A search optimization module processes input words such as “create”, “creates”, and “creating” to ensure consistent retrieval across grammatical variations. Design a morphology-based normalization approach focusing on inflectional transformations such as tense and aspect.
Identify suffix patterns and map all variations to a common base form without altering meaning.
Apply the process to extract root forms and classify each transformation.
Determine the morphological breakdown and normalized output for each word.

- Develop a Python program to implement an inflectional morphology-based normalization module for the input words "create", "creates", and "creating". The program should recognize grammatical suffixes corresponding to different inflectional forms, identify the underlying root word by applying appropriate normalization rules, classify each input according to its grammatical feature (e.g., base form, third-person singular, present participle), and produce a structured output containing the original word, identified suffix, grammatical category, extracted root, normalized base form, and the final normalized representation to support efficient search optimization and information retrieval.

Question 5: Inflectional Normalization of “create”, “creates”, and “creating”

Approach: These forms share the lemma “create”. “Creates” takes “-s” for the third-person singular present tense. “Creating” takes “-ing” to form the present participle/gerund and expresses an ongoing action or an activity. These are inflectional forms, so normalization to “create” does not change the core lexical meaning.

Python Program:

```
words = ["create", "creates", "creating"]
analysis = {
   "create": ("", "base form", "create"),


"creates": ("-s", "third-person singular present", "create"),
 "creating": ("-ing", "present participle/gerund", "create")
}
print(f'{"Word":<12} {"Suffix":<10} {"Grammatical category":<34} '
     f'Normalized base')
for word in words:


 suffix, category, root = analysis[word]


 print(f'{word:<12} {suffix:<10} {category:<34} {root}')
```

Output: -

```
========================================== RESTART: C:/Users/Reddy/Docu
Word        Suffix     Grammatical category            Normalized base
-----------------------------------------------------------------------
create                 base form                       create
creates     -s         third-person singular present   create
creating    -ing       present participle/gerund       create
```

Result: - The program has successfully processed the word list, extracted the grammatical suffixes, and mapped each inflected form back to its normalized base. All columns have been aligned perfectly for optimal readability.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____